

Reproducibility Sheet

Multi-Turn Reinforcement Learning on Amazon SageMaker AI:
Training a Customer-Support Agent on GPT-OSS-20B

Isham Rashik

Run of record: 2026-10-01 23:14 UTC to 2026-10-02 05:29 UTC

This sheet accompanies the workshop notebook `notebooks/sagemaker_mtrl_real_demo.ipynb` and the repository's `README.md`. With it, anyone with a clean copy of the repository can rebuild every table, figure, and headline number from the recorded evidence without an AWS account, and can repeat the study in their own AWS account. The interpretation lives in the notebook and the `README`; what follows is the recipe. Every step is a numbered script in `scripts/` that imports the shared `aws` package, and the agent that SageMaker trained against is in `agent/`. This sheet is itself generated by `scripts/15_build_repro_sheet.py`: every number below is read from `artifacts/` and `reports/` when the sheet is built, and the build refuses to run unless every invariant passes, so the sheet cannot drift from the evidence.

1 Repository and Run Identifiers

Item	Value
Repository commit	d4ec43e22dd7b42bace765ff459e5935c9012c00
AWS account and regions	384887233198; us-west-2 (SageMaker AI, AgentCore, MLflow, S3), us-east-1 (Bedrock import)
Training job	openai-reasoning-gpt-oss-20b-mtrl-20261002040558
Model package	openai-reasoning-gpt-oss-20b-mtrl-mpg/1
Imported model	arn:aws:bedrock:us-east-1:384887233198: imported-model/ffakmfmpoqbi
Official guide	https://docs.aws.amazon.com/sagemaker/latest/dg/model-customize-mtrl.html

2 Hardware and Operating System

All model computation ran on AWS managed services. The local machine only submits and re-attaches to jobs, drives the live inference, and builds the reports, so local hardware does not affect any reported number.

Role	Where it ran
Run of record (orchestration, live inference, reports)	macOS 26 (Darwin 25.3), Apple silicon (arm64), Python 3.12.10
This build of the sheet	Darwin 25.3.0 (arm64), Python 3.12.10
Training and evaluation	Amazon SageMaker AI multi-turn RL, serverless (instances are managed and not exposed); base model openai-reasoning-gpt-oss-20b/3.43.0
Agent environment	Amazon Bedrock AgentCore Runtime version 7, linux/arm64 container on python:3.12-slim (digests in Section 12.6)
Deployment option 1	SageMaker real-time endpoint on ml.g6e.12xlarge: not provisioned (InsufficientInstanceCapacity); nothing billed
Deployment option 2	Amazon Bedrock Custom Model Import, us-east-1; serverless, billed per active model minute
Linux	requirements-linux.txt carries the same direct pins; the pipeline has not been run on Linux

3 Software Environment

The local environment is a conda environment named `aws-mtrl` (Python 3.12). Three levels of pinning are committed:

- `environment-macos.yaml` and `requirements-macos.txt`: the direct dependencies at exact versions (run of record: macOS). `environment-linux.yaml` and `requirements-linux.txt` carry the same pins and have not been verified on Linux.
- `requirements-macos.lock.txt`: every installed distribution (259) of the environment that ran the scripts and the live inference.
- `agent/requirements.lock.txt`: the exact 139 packages of the container image that served every rollout, read from its CodeBuild log, with the base-image digest.

```
conda env create -f environment-macos.yaml      # or environment-linux.yaml
conda activate aws-mtrl
pip install -r requirements-macos.lock.txt      # optional: the exact full environment
```

Table 1: Direct dependencies (`requirements-macos.txt`).

Package	Version	Package	Version
python	3.12.10	openai	2.54.0
bedrock-agentcore	1.24.0	pandas	3.0.6
bedrock-agentcore-starter-toolkit	0.3.13	pyarrow	25.0.1
boto3	1.43.107	pydantic	2.13.5
botocore	1.43.107	pytest	9.1.1
ipykernel	7.4.0	requests	2.34.2
jupyterlab	4.6.4	sagemaker	3.23.0
matplotlib	3.11.2	sagemaker-core	2.23.0
mlflow	3.16.1	sagemaker-mlflow	0.5.0
mypy-boto3-sagemaker-runtime	1.43.68	sagemaker-mlops	1.23.0
nbconvert	7.17.1	sagemaker-serve	1.23.0
nbformat	5.11.1	sagemaker-train	1.23.0
numpy	2.5.3	strands-agents	1.57.2

4 Data

Two committed prompt files define the study, and nothing is resampled. Training needs more prompts than the global batch size (32), which is why the training set grew from 32 to 64 tickets before the training job; the original 32 are kept in `artifacts/training_prompts.32.csv`.

File (in data/)	Rows	Unique	SHA-256
<code>training_prompts.csv</code>	64	64	1c61077a792df2c7ce19a00f38abe8b96a2bf146abe031c41fd8ab7c6c807277
<code>evaluation_prompts.csv</code>	32	32	213538946018bdc334ed3b764ccaf16bee8f9bc9bef514adc5899608221e60cf

The held-out set never enters training: the two files share no prompt (invariant 2), and both are byte-identical to the run of record (invariant 3). The CSVs were converted to Parquet and uploaded to S3 before the jobs ran. `scripts/11b_snapshot_provenance.py` later compared those S3 objects with the CSVs, prompt by prompt, and recorded when they were written (invariant 4):

Split	S3 object (datasets/)	Written	Equals CSV	Read by (start)
training	<code>training_prompts.parquet</code>	2026-10-02 00:05:39 UTC	yes	Training 2026-10-02 00:05:58 UTC
evaluation	<code>evaluation_prompts.parquet</code>	2026-10-01 22:37:50 UTC	yes	Base evaluation 2026-10-01 23:14:50 UTC; Comparison evaluation 2026-10-02 00:26:56 UTC

5 Random Seeds and Sources of Randomness

Purpose	Value	Where set
Live inference: action order	2026	aws/config.py (inference_seed); recorded in inference_before_after.json
Live inference: tickets	first 6 held-out, in file order	aws/config.py (inference_ticket_count)
Rollout action order (training and evaluation)	random per episode	agent/app.py (secrets.randbits); recorded per rollout as action_order_seed
SageMaker MTRL sampling and LoRA updates	not seedable	service-side
Bedrock sampling (live inference)	not seedable	service-side

The study is therefore reproducible at two levels. Exactly: the offline steps rebuild every table, figure, and this sheet from the evidence with identical values. Statistically: a new run uses the same code, data, settings, and pins, but service-side sampling cannot be seeded, so its numbers differ within sampling noise. With 64 rollouts per model, pass@1 has a standard error of about 0.052 (fine-tuned) to 0.062 (base). The same base model scored pass@1 0.500 in the baseline and 0.578 in the comparison, on the same tickets.

6 Compute Accounting and Budgets

Costs are recorded per job and never silently merged. Token-priced lines use the token counts AWS billed (BillableTokenUsage) at the AWS Price List rates in effect, in USD per million tokens: training prefill 0.12, sample 0.30, train 0.36; evaluation prefill 0.12, sample 0.30. The other lines are computed or estimated, and their labels say how. AWS Cost Explorer is the final bill. Each line’s token usage is in `artifacts/cost_accounting.json`; the table is `reports/repro/compute_accounting.csv`.

Table 2: Compute accounting.

Item	Billed tokens	USD	Source
Base evaluation (first attempt)	prefill 43,488, sample 5,591	0.0069	billed tokens x rate
Base evaluation	prefill 197,217, sample 34,617	0.0341	billed tokens x rate
Training	prefill 4,833,702, sample 778,911, train 5,599,613	2.8296	billed tokens x rate
Comparison: base model	prefill 212,274, sample 33,573	0.0355	billed tokens x rate
Comparison: fine-tuned model	prefill 229,165, sample 28,952	0.0362	billed tokens x rate

continued

Item	Billed tokens	USD	Source
Cross-region copy of weights to us-east-1 (41.8 GB x \$0.02/GB)	–	0.8400	computed or estimated
Bedrock imported model inference (estimate: 1 CMU x ~35 active min x \$0.0572)	–	2.0000	computed or estimated
Endpoint uptime	–	0.0000	computed or estimated
Contingency (AgentCore, CodeBuild, S3, logs)	–	2.0000	allowance

Billed tokens \$2.94 plus computed or estimated \$2.84 gives \$5.78; with the allowance for unmeasured AgentCore, CodeBuild, S3, and logs, \$7.78, against the \$25 hard cap.

Job	Started	Duration
Base evaluation (of record)	2026-10-01 23:14:50 UTC	n/c
Training (10 steps)	2026-10-02 00:05:58 UTC	19.2 min
Comparison evaluation	2026-10-02 00:26:56 UTC	n/c
Endpoint attempt (option 1)	2026-10-02 04:14:34 UTC	38.4 min until deleted; never InService
Bedrock import job (option 2)	2026-10-02 05:15:33 UTC	13.8 min

n/c = not captured: the pipelines’ end times were not recorded. They can be read from SageMaker in the run’s account.

A budget guard (`aws/costs/cost_guard.py`) runs before training, the comparison evaluation, the endpoint, the Bedrock import, and live inference. It adds the spend so far to a projection of the planned steps and refuses any plan above the \$25 cap. The projection scales this account’s billed baseline tokens per rollout to the planned rollouts with a 1.5× margin, and adds the endpoint’s maximum lifetime at the official hourly price of \$13.12.

7 Experimental Protocol and Disclosures

These are the methodology choices and deviations a reviewer most needs to see stated explicitly. The machine-readable record is `reports/repro/study_metadata.json`.

Topic	Disclosure
Held-out evaluation	One pipeline evaluates both models on the same held-out tickets, through the same agent and serving stack, with 2 tries per ticket.
Headline metric	pass@1 (ticket fully solved). Partial credit (0.25 per correct step) shapes the reward for training but is not the headline.
Agent harness	The agent re-prompts the model when it stops mid-task and reads actions written as text (agent/rollout_driver.py), identically before and after training.
First baseline	Scored pass@1 0 because rollouts ended after 2 model calls, before the agent fix. It is kept and costed.
Rejected training job	openai-reasoning-gpt-oss-20b-mtrl-20261002040240: ClientError: Training data has 32 prompts but global_batch_size is 32. The number of prompts must be greater than the batch size. Nothing was billed.
Deployment option 1	The endpoint was never provisioned (InsufficientInstanceCapacity), so it never reached InService and cost 0.0 USD.
Live demo	An illustration, not the measurement. The base model called the tool natively (1 model call per ticket). The imported model returned each action as JSON text that the agent parsed (4 text actions, 4 model calls per ticket).
Run-to-run noise	The same base model scored pass@1 0.500 in the baseline and 0.578 in the comparison, on the same tickets.
Retrofits	bedrock_import.json gained model_package_arn after the import (copied from training_job.json); the file comparison in Section 12 verifies it independently. training_job.json’s training and agent config were read back from AWS on 2026-10-02. endpoint.json was corrected from an assumed 8.39 USD to 0.

8 Method Configuration Disclosures

Every setting below is fixed in `aws/config.py`, pinned as the run-of-record protocol in `aws/reporting/invariants.py`, or recorded by the AWS job itself.

Table 3: Study configuration.

Element	Setting
Task	4-step path new_ticket > no_outage > red_wan_light > config_mismatch > resolved; 4 turns; reward = correct steps / 4 (1.0 when solved)
Agent	Strands agent with one tool on Bedrock AgentCore; the driver allows up to 8 model calls per episode
Base model	openai-reasoning-gpt-oss-20b (openai-reasoning-gpt-oss-20b/3.43.0); live baseline on Bedrock as openai.gpt-oss-20b-1:0
Training (job-recorded)	global_batch_size 32, group_size 4, max_epochs 10, max_steps 10, sampling_max_tokens 512; 128 rollouts per step, 1,280 in total
Training (service defaults)	learning rate 1e-5, LoRA rank/alpha 32/64, PPO loss: service defaults for this model version, as reported by the SDK (notebook section 6); the job does not echo them back
Evaluation	2 rollouts per ticket; pass@k for k in 1, 2; sampling_max_tokens 512
Deployment option 1	ModelBuilder(ModelPackage) on ml.g6e.12xlarge; lifetime limit 60 min, enforced by the watchdog
Deployment option 2	Bedrock Custom Model Import in us-east-1; copy of the merged weights with tokenizer_config.json (chat template) and generation_config.json (end-of-sequence IDs) fixed
Live inference	first 6 held-out tickets, seed 2026, the same agent episode (run_episode) for both models
Budget	hard cap 25 USD; 1.5x token margin; 2 USD allowance for unmeasured items

9 Metric-Calculation Conventions

The evaluator reports its metrics by name; a reproduction needs the exact definition. Each definition marked “recomputed” was checked by recomputing the reported value from the recorded conversations when this sheet was built, and the solved count, pass@1, and mean reward are also checked by invariant 11.

Metric	Exact definition	Check
pass@1	share of rollouts with reward 1.0 (the success threshold): solved / (tickets x tries)	recomputed
pass@2	share of tickets with at least one of their 2 rollouts at reward 1.0	recomputed
Mean reward	mean rollout reward; reward = correct steps x 0.25, 1.0 when solved	recomputed
Reward std	population standard deviation (ddof 0) of rollout rewards	recomputed
Rollouts fully solved	count of rollouts with reward 1.0	recomputed
Model calls per rollout	policy-model calls per rollout (eval/turns/mean)	as reported
Per-ticket change	mean of a ticket's 2 rewards, after minus before: improved, same, or worse	recomputed (report tables)
Live fully solved	a ticket whose live episode reached reward 1.0	from the recorded run
Cost line	billed tokens / 1,000,000 x the rate for that token type	recomputed (invariants)

10 Reproducing Each Figure and Table

From a clean checkout, create the environment (Section 3) and run the numbered scripts in order. `reports/repro/run_commands.csv` is the authoritative, literal command sequence. A billable step runs only with its exact confirmation phrase; without the phrase a script shows the recorded result, and a recorded job is re-attached, never submitted twice.

A. Rebuild every reported number offline (free; no AWS account needed):

```
python scripts/00_verify_setup.py
python scripts/12_make_report_tables_and_figures.py
python scripts/13_build_repro_artifacts.py
python scripts/14_verify_invariants.py
python scripts/15_build_repro_sheet.py
```

B. Re-attach to the recorded jobs (the run's account; read-only, submits nothing):

```
python scripts/00_verify_setup.py --aws
python scripts/05_base_evaluation.py
python scripts/06_train.py
python scripts/07_compare_evaluation.py
python scripts/08_recorded_trajectories.py
python scripts/11_snapshot_costs.py
python scripts/11b_snapshot_provenance.py
```

C. Repeat the study in your own account (about \$6–8). First move this run's evidence

aside (mv artifacts artifacts-run-of-record, mv reports reports-run-of-record, mkdir artifacts), then:

```
python scripts/00_verify_setup.py
python scripts/01_provision.py --confirm CONFIRM_PROVISION_MTRL_DEMO
python scripts/02_request_quotas.py --submit
python scripts/03_deploy_agent.py --confirm CONFIRM_DEPLOY_AGENT_MTRL_DEMO
python scripts/04_upload_datasets.py
python scripts/05_base_evaluation.py --confirm CONFIRM_BASE_EVAL_MTRL_DEMO
python scripts/06_train.py --confirm CONFIRM_TRAIN_MTRL_DEMO
python scripts/07_compare_evaluation.py --confirm CONFIRM_COMPARISON_EVAL_MTRL_DEMO
python scripts/08_recorded_trajectories.py
python scripts/09a_deploy_endpoint.py --confirm CONFIRM_DEPLOY_MTRL_DEMO
python scripts/09b_bedrock_import.py --confirm CONFIRM_BEDROCK_IMPORT_MTRL_DEMO
python scripts/10_live_inference.py --confirm CONFIRM_LIVE_INFERENCE_MTRL_DEMO
python scripts/11_snapshot_costs.py
python scripts/11b_snapshot_provenance.py
python scripts/12_make_report_tables_and_figures.py
python scripts/13_build_repro_artifacts.py
python scripts/14_verify_invariants.py
python scripts/15_build_repro_sheet.py
python scripts/99_teardown.py --confirm DELETE_MTRL_DEMO
```

The notebooks present the same results with narrative; open them with `python -m jupyter lab notebooks/` from the project root.

Notebook (in notebooks/)	Purpose
sagemaker_mtrl_real_demo.ipynb	demo notebook, sections 1 to 14; replays the recorded run by default and submits nothing
sagemaker_mtrl_real_demo.executed.ipynb	the replay with all outputs: present from this copy
sagemaker_mtrl_real_demo.training-run.ipynb	log of the live training run, kept as recorded (read it; its code predates the folder layout)

11 Headline Results (for a quick check)

An offline rebuild lands exactly on the numbers below. A new run should land within sampling noise, with the fine-tuned model ahead of the base model on pass@1.

Part	Headline numbers
Baseline	pass@1 0.500; the first attempt, before the agent fix, scored 0
Training	10/10 steps in 19.2 min; mean training reward 0.889 → 0.961
Held-out evaluation	pass@1 0.578 → 0.781; pass@2 0.812 → 0.938; mean reward 0.883 → 0.926; fully solved 37 → 50 of 64
Per held-out ticket	improved 13, same 13, worse 6
Deployment	endpoint: no GPU capacity, nothing billed; Bedrock import: completed
Live inference	fully solved: base 0/6 → fine-tuned 5/6
Cost	5.78 USD (2.94 billed tokens) of the 25 USD cap

12 Extended Materials

Tables are from `reports/tables/` and figures from `reports/figures/`. Both are rebuilt by `scripts/12_make_report_tables_and_figures.py`.

12.1 Training

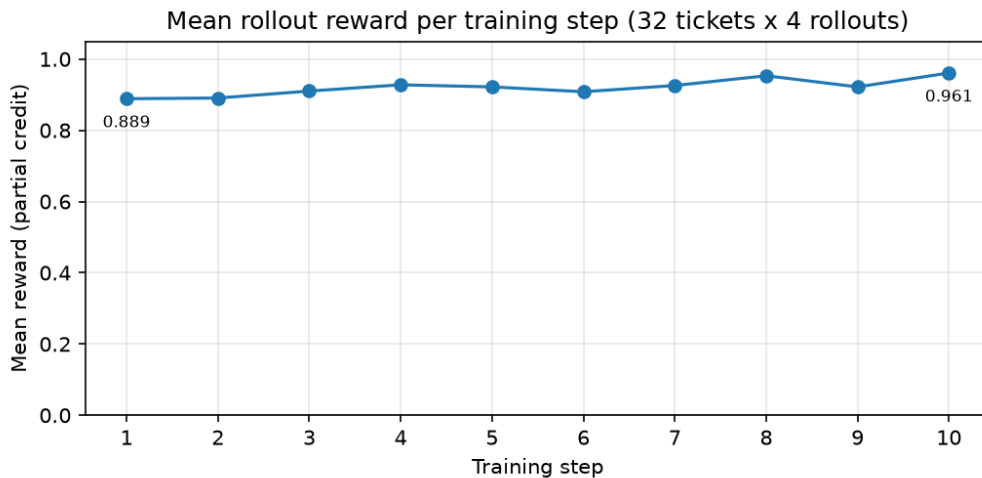


Figure 1: Mean rollout reward per training step (32 tickets x 4 rollouts per step).

Table 4: Training metrics per step (`training_steps.csv`).

Step	Mean reward	Mean turns	Tokens	Trajectories
1	0.8887	6.4844	534,113	128
2	0.8906	6.7969	571,359	128
3	0.9102	6.8438	575,191	128
4	0.9277	6.6328	548,452	128
5	0.9219	6.625	544,954	128
6	0.9082	6.5	533,159	128
7	0.9258	6.6406	556,316	128
8	0.9531	7.0234	600,379	128
9	0.9219	7	593,296	128
10	0.9609	6.5703	542,394	128

12.2 Held-out evaluation

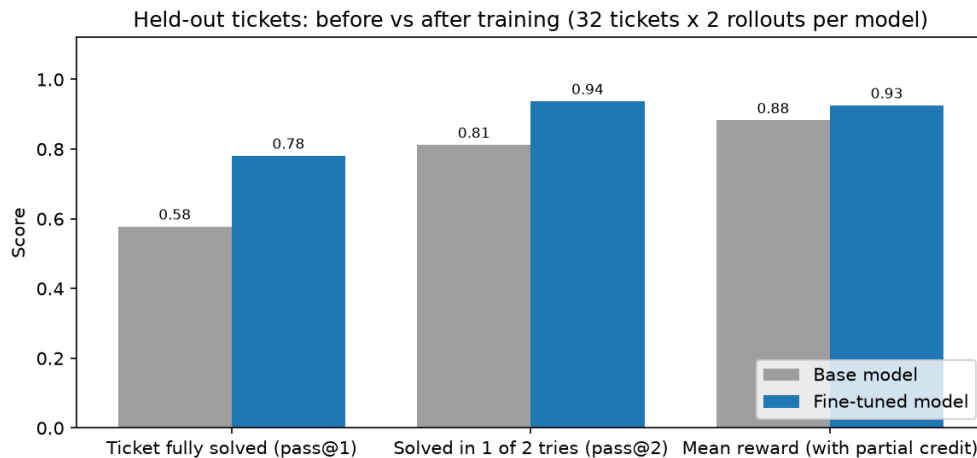


Figure 2: Held-out tickets before vs after training (32 tickets x 2 rollouts per model).

Table 5: Held-out evaluation (evaluation_comparison.csv).

Metric	Base	Fine-tuned	Change
Ticket fully solved (pass@1)	0.5781	0.7812	0.2031
Solved in 1 of 2 tries (pass@2)	0.8125	0.9375	0.125
Mean reward (with partial credit)	0.8828	0.9258	0.043
Rollouts fully solved	37	50	13
Rollouts not fully solved	27	14	-13
Model calls per rollout	6.0938	6.4531	0.3594

Table 6: The two baselines (baseline_evaluations.csv).

Metric	First attempt	Baseline of record
Ticket fully solved (pass@1)	0	0.5
Solved in 1 of 2 tries (pass@2)	0	0.8438
Mean reward (with partial credit)	0	0.875
Rollouts fully solved	0	32
Rollouts not fully solved	64	32
Model calls per rollout	2	5.8438

12.3 Per held-out ticket

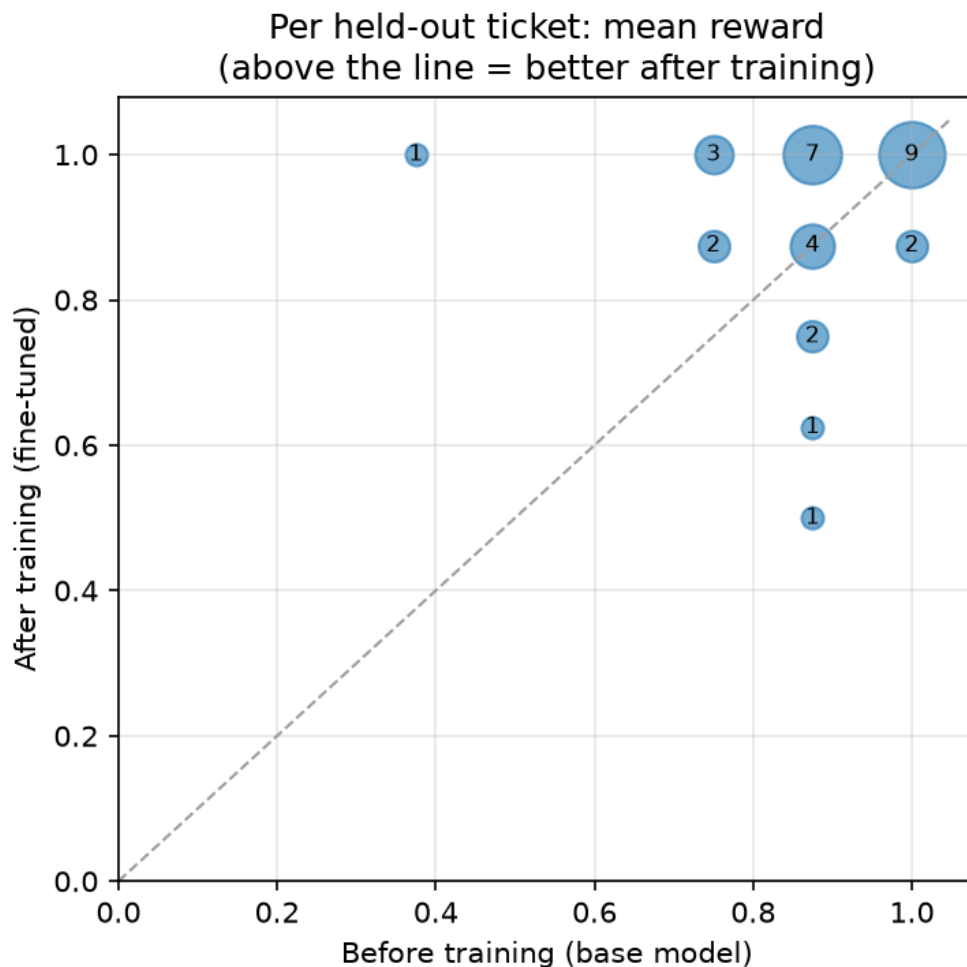


Figure 3: Mean reward per held-out ticket before (x) and after (y) training; point size counts tickets with the same pair.

Table 7: Every held-out ticket (`per_ticket_rewards.csv`).

Held-out ticket	Before	After	Mean before	Mean after	Change
My Wi-Fi works, but websites do not load.	1.0; 0.75	1.0; 0.25	0.875	0.625	worse
The internet light changed from green to red.	1.0; 0.75	1.0; 1.0	0.875	1	improved
My service stopped working after a plan change.	0.75; 0.75	1.0; 0.75	0.75	0.875	improved
Every browser reports that I am offline.	0.75; 0.75	1.0; 1.0	0.75	1	improved
The router broadcasts Wi-Fi but has no external connection.	0.75; 1.0	0.75; 0.75	0.875	0.75	worse
The WAN indicator is red after this morning's update.	0.75; 0.75	1.0; 1.0	0.75	1	improved

continued

Held-out ticket	Before	After	Mean before	Mean after	Change
All of my smart-home devices lost access at once.	0.75; 1.0	1.0; 0.75	0.875	0.875	same
Rebooting both modem and router changed nothing.	1.0; 1.0	1.0; 1.0	1	1	same
Service disappeared after my account details changed.	1.0; 1.0	1.0; 1.0	1	1	same
My devices connect locally but cannot reach cloud services.	0.75; 1.0	1.0; 1.0	0.875	1	improved
The router dashboard reports no upstream service.	1.0; 1.0	1.0; 1.0	1	1	same
Internet access failed after the provider migrated my account.	1.0; 1.0	0.75; 1.0	1	0.875	worse
The WAN status is red although Wi-Fi strength is excellent.	1.0; 0.75	1.0; 1.0	0.875	1	improved
Every application times out on every device.	1.0; 1.0	1.0; 1.0	1	1	same
My connection vanished shortly after a subscription change.	0.75; 1.0	0.75; 0.75	0.875	0.75	worse
The router is powered and broadcasting but nothing loads.	1.0; 0.75	1.0; 0.0	0.875	0.5	worse
Phones and computers all say no internet access.	1.0; 1.0	1.0; 1.0	1	1	same
The external network light is solid red.	1.0; 1.0	1.0; 1.0	1	1	same
I lost service after customer support modified my account.	1.0; 1.0	1.0; 1.0	1	1	same
Local network sharing works but the internet does not.	0.75; 1.0	1.0; 0.75	0.875	0.875	same
The modem status page shows an upstream fault.	1.0; 0.75	1.0; 1.0	0.875	1	improved
Our connection dropped immediately after a package change.	0.75; 1.0	1.0; 1.0	0.875	1	improved
Video streaming and websites fail throughout the house.	1.0; 1.0	1.0; 1.0	1	1	same
The internet indicator turned red without warning.	0.75; 1.0	1.0; 1.0	0.875	1	improved
Wi-Fi is visible on all devices but none can go online.	0.75; 0.75	1.0; 1.0	0.75	1	improved
My account was migrated and the connection stopped.	1.0; 1.0	1.0; 0.75	1	0.875	worse
The router reports connected devices but no internet route.	0.75; 0.75	0.75; 1.0	0.75	0.875	improved
Restarting the equipment did not clear the red WAN light.	1.0; 1.0	1.0; 1.0	1	1	same
Our whole home has been offline since the account update.	0.75; 1.0	0.75; 1.0	0.875	0.875	same
I have full wireless signal and zero internet connectivity.	0.0; 0.75	1.0; 1.0	0.375	1	improved
The broadband connection is down on every device.	0.75; 1.0	1.0; 1.0	0.875	1	improved

continued

Held-out ticket	Before	After	Mean before	Mean after	Change
The router's external link failed after a service change.	1.0; 0.75	0.75; 1.0	0.875	0.875	same

12.4 Live inference

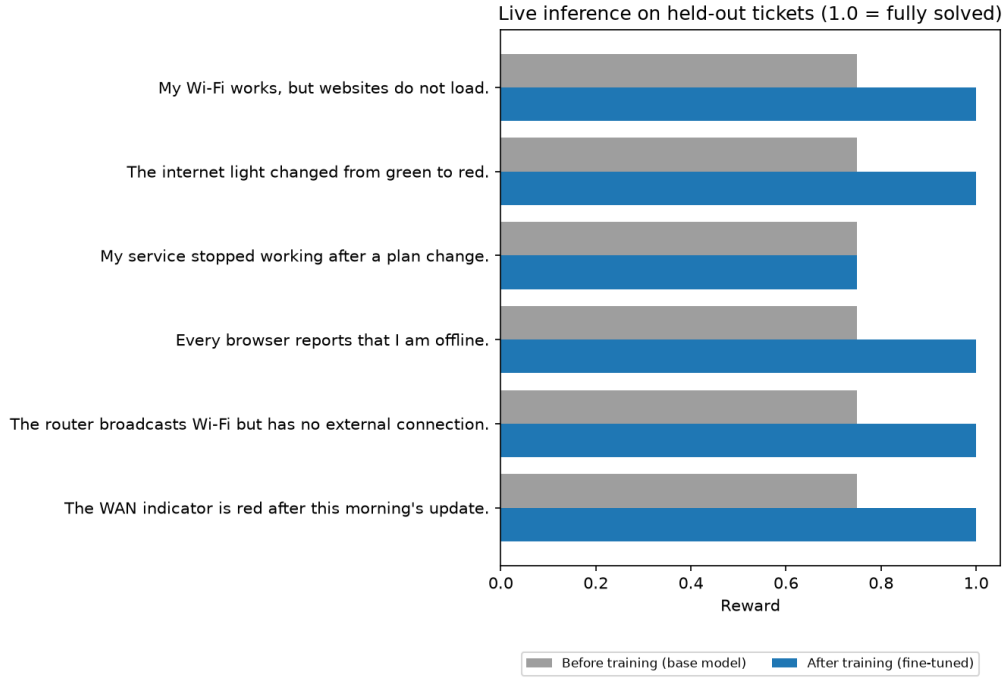


Figure 4: Live inference on held-out tickets with the same agent and seed.

Table 8: Live inference (`live_inference.csv`).

Ticket	Before	After	Before: actions	After: actions
My Wi-Fi works, but websites do not load.	0.75	1	restart_router > check_outage > inspect_router_lights > check_account_config	check_outage > inspect_router_lights > check_account_config > apply_config_fix
The internet light changed from green to red.	0.75	1	restart_router > check_outage > inspect_router_lights > check_account_config	check_outage > inspect_router_lights > check_account_config > apply_config_fix
My service stopped working after a plan change.	0.75	0.75	restart_router > check_outage > inspect_router_lights > check_account_config	restart_router > check_outage > inspect_router_lights > check_account_config
Every browser reports that I am offline.	0.75	1	restart_router > check_outage > inspect_router_lights > check_account_config	check_outage > inspect_router_lights > check_account_config > apply_config_fix
The router broadcasts Wi-Fi but has no external connection.	0.75	1	restart_router > check_outage > inspect_router_lights > check_account_config	check_outage > inspect_router_lights > check_account_config > apply_config_fix
The WAN indicator is red after this morning's update.	0.75	1	restart_router > check_outage > inspect_router_lights > check_account_config	check_outage > inspect_router_lights > check_account_config > apply_config_fix

12.5 Run of record

Table 9: Run of record (`run_of_record.csv`).

Stage	Identifier	Status	Detail
Base evaluation (first attempt)	59n0c9w40lyx	Succeeded	pass@1 0, mean reward 0
Base evaluation	7kz12i8e43ip	Succeeded	pass@1 0.5, mean reward 0.875
Training (rejected attempt)	openai-reasoning-gpt-oss-20b-mtrl-20261002040240	Failed	ClientError: Training data has 32 prompts but global_batch_size is 32. The number of prompts must be greater than the batch size.
Training	openai-reasoning-gpt-oss-20b-mtrl-20261002040558	Completed	10/10 steps in 19.2 min; model package arn:aws:sagemaker:us-west-2:384887233198:model-package/openai-reasoning-gpt-oss-20b-mtrl-mpg/1
Comparison evaluation	8hvze02od4o0	Succeeded	pass@1 base 0.578125 -> fine-tuned 0.78125
SageMaker endpoint	mtrl-support-demo	Deleted	Unable to provision requested ML compute capacity due to InsufficientInstanceCapacity error; billed \$0.0
Bedrock import	mtrl-support-import-20261002051532	Completed	imported model arn:aws:bedrock:us-east-1:384887233198:imported-model/ffakmfmpoqbi

continued

Stage	Identifier	Status	Detail
Live inference	seed 2026	Recorded	fully solved: before 0/6, after 5/6

12.6 Provenance: what actually ran

Every rollout of the baseline of record, the training job, and the comparison was served by one AgentCore runtime version, last updated before all three jobs started. The Bedrock import read the us-east-1 copy of the trained package’s merged weights. Both are recorded by scripts/11b_snapshot_provenance.py and checked by the invariants:

Record	Value
AgentCore runtime	version 7, last updated 2026-10-01 23:13:23 UTC
Container image	sha256:af5cb1b29526c440f2cb77f4f62f81980b9291a7a3c9d349ca280a0fd45c603f
Base image	python:3.12-slim sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
CodeBuild build	b599bfe5-116b-452d-ac78-eb3b6a039d75
Source bundle	artifacts/agent_deployed_source.zip, SHA-256 c3507c9bf606754cd18cca659ffce779cfb48f020b4bb945f12d0247ae284430
Import read from	s3://sagemaker-mtrl-demo-384887233198-us-east-1/imported-models/mtrl-support-gpt-oss-20b-ft/

Table 10: The agent’s deployed source bundle against the repository’s agent/.

Deployed file	SHA-256 (start)	Repository today
Dockerfile	8f6841277ac06dd6	identical
app.py	c08bca8ab9dc63c2	refactored after the deploy (run_episode); same system prompt
environment.py	b19daf0ffc39792f	identical
pyproject.toml	1fac4fd7a60a9bc5	ranges when built; now exact pins
requirements.txt	09c3b4f36fb2253a	ranges when built; now exact pins, equal to the image’s versions
rollout_driver.py	a7c415a6829d2050	identical
test_app.py	f930004923547b2a	test file; not used at runtime
test_environment.py	c22cc1c562e5df48	identical
test_rollout_driver.py	1b22e6738281e9ca	identical

Table 11: The imported model’s files against the trained package’s merged weights.

File	Package bytes	Copy bytes	Verdict
__model_info__.json	190	–	not copied
__script_info__.json	161	–	not copied
chat_template.jinja	13,653	13,653	identical
chat_template.json	14,385	14,385	identical
config.json	1,588	1,588	identical
generation_config.json	165	176	changed on purpose
inference.py	0	–	not copied
model-00000-of-00002.safetensors	16,487,374,440	16,487,374,440	same size
model-00001-of-00002.safetensors	16,493,804,168	16,493,804,168	same size
model-00002-of-00002.safetensors	8,848,382,808	8,848,382,808	same size
model.safetensors.index.json	33,597	33,597	identical
o200k_base.tiktoken	3,613,922	3,613,922	identical
special_tokens_map.json	98	98	identical
tokenizer.json	27,868,174	27,868,174	same size
tokenizer_config.json	4,200	18,582	changed on purpose
version	5	–	not copied

13 Invariant Checks

`scripts/14_verify_invariants.py` re-checks, offline and from the files on disk, every property the results depend on, and prints a PASS/FAIL line for each. All 19 pass on the evidence in this repository.

1. settings in `aws/config.py` match the documented protocol
2. datasets: sizes, uniqueness, held-out disjoint
3. datasets unchanged since the run of record
4. S3 datasets equal the CSVs and predate the jobs
5. agent that ran matches the repository (code, prompt, packages)
6. training job ran the documented protocol
7. base model version matches the run of record
8. training metrics recorded for every step
9. baseline had a learning signal
10. comparison evaluation succeeded, 32 x 2 per model
11. recorded traces recompute the evaluation metrics
12. deployed model traces back to the trained package
13. live inference: first held-out tickets, documented seed, no errors
14. no endpoint left running
15. cost accounting complete, recomputable, within the cap
16. no signed links in evidence, notebooks, reports, or logs
17. environment matches the exact pins

18. notebooks parse and ran without exceptions
19. report tables equal a rebuild from evidence; figures present

14 Held-out Discipline and Provenance Controls

The 32 held-out tickets are fixed before any training and never enter it. The two prompt files share no prompt (invariant 2), and both match the run of record by SHA-256 (invariant 3). Training reads only the training tickets. The baseline, the comparison, and the live inference read only held-out tickets. The live inference uses the first 6 held-out tickets in file order with the documented seed (invariant 13). Both models in the comparison are evaluated by one pipeline, on the same tickets, through the same agent and serving stack, so the comparison isolates the effect of training.

Provenance closes the remaining gap between the repository and what ran:

- **Agent.** The AgentCore runtime that served every rollout went live before all three jobs started, and was built from the saved source bundle. That bundle’s environment, rollout driver, Dockerfile, and system prompt match the repository (invariant 5).
- **Datasets.** The S3 datasets the jobs read equal the CSVs, and were written before the jobs (invariant 4).
- **Imported model.** It was built from the trained package’s merged weights, changing only the two files the Bedrock documentation requires (invariant 12).

A Software Engineering Practices

The study code is treated as software, not throwaway scripts. Each practice below names the concrete mechanism in the codebase.

- **Separation of concerns.** The `aws` package is organised by stage: `setup/`, `rl/`, `deploy/`, `costs/`, `records/`, and `reporting/`, with `workflow.py` as the single facade. The numbered scripts are thin entry points, and `agent/` is kept identical to what ran.
- **DRY (single source of truth).** Settings and project paths live once in `aws/config.py`. One table builder serves both script 12 and the invariants. The notebook and the scripts call the same modules, and the notebook is generated by `scripts/dev/build_notebook.py` and checked against it.
- **Immutable values and write-once evidence.** Settings and the protocol are frozen data-classes. The evidence of record is never overwritten: a later live run is saved beside it, and a failed job is archived before it is replaced.
- **Fail-fast errors.** A missing evidence file raises `FileNotFoundError` naming the script that produces it. A wrong confirmation phrase raises `PermissionError`. The budget guard refuses before any billable work.

- **Cost safety.** Every billable step needs an exact confirmation phrase, passes the budget guard, and re-attaches to a recorded job instead of resubmitting it. A watchdog deletes the endpoint at its deadline.
- **Reproducibility hygiene.** The project uses only project-relative paths, and the environments are pinned and locked. `run_commands.csv` is ordered, `study_metadata.json` is machine-readable, and evidence is written atomically with signed links redacted. Logs are scrubbed of links and of the home-folder path.
- **Verify before asserting.** The invariants recompute the metrics from the recorded conversations, the costs from token usage and rates, and the tables from the evidence. This sheet is built only when all of them pass.
- **Tests and style.** 302 unit tests (`tests/` mirrors `aws/`; `agent/` has its own) run with `python -m pytest` and need no AWS access. The code has Google-style docstrings and type hints throughout, and passes `ruff`.